

Ordinary Differential Equations (ODEs)

Outline

- Introduction
- Initial Value Problems (IVP)
 - Non-stiff systems
 - Higher order ODEs
 - Systems of ODEs
 - Stiff systems
- Boundary Value Problems (BVP)
- Engineering Applications

Introduction

- What is an ODE ?

- An ODE is a differential equation containing only one independent variable

$$\frac{dy}{dx} = y^2 + xy + c$$

- In the above equation the independent variable is x
- Many civil engineering problems require solution of ODEs
 - Environmental/Chemical Reactor Kinetics
 - Structural vibration

Initial Value Problems (IVPs)

Initial Value Problems (IVP's)

- Non-stiff systems
 - Forward Euler
 - Heun's method
 - 4th order Runge-Kutta
 - Extension to system of ODEs
 - Extension to higher order ODEs
 - Matlab functions: ode45, ode23
- Stiff systems
 - Implicit Euler
 - Multi-step methods
 - Semi-implicit methods
 - Matlab functions: ode15s, ode23s

What is an initial value problem ?

- For n -order ODE or n -system ODE:
 - all n conditions specified at the same fixed value of independent variable (i.e., $x = x_0$)

n^{th} order ODE:

$$\frac{d^n y}{dx^n} = f\left(x, y, \frac{dy}{dx}, \dots, \frac{d^{n-1}y}{dx^{n-1}}\right)$$

initial conditions:

$$x = x_0, y = a_1; \quad x = x_0, \frac{dy}{dx} = a_2; \quad x = x_0, \frac{d^2 y}{dx^2} = a_3;$$

n - system ODE :

$$\frac{dy_1}{dx} = f_1(x, y_1, y_2, \dots) \quad x = x_0, y_1 = a_1$$

$$\frac{dy_2}{dx} = f_2(x, y_1, y_2, \dots) \quad x = x_0, y_2 = a_2$$

$$\vdots$$

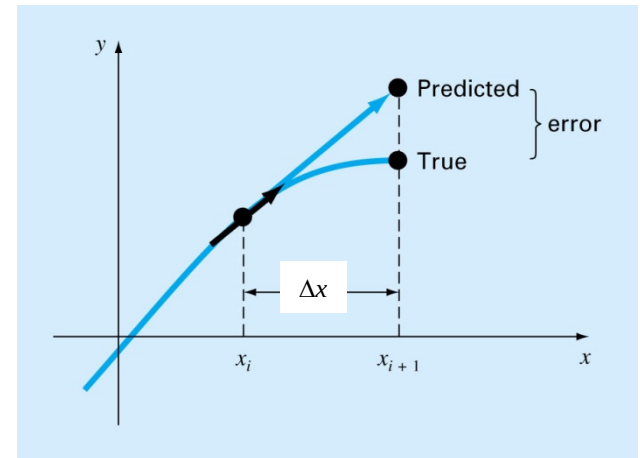
Numerical Solution of IVP ODEs

- Euler method
- Heuns method
- Mid-point method
- 4th order Runge-Kutta method

Forward Euler Method

- In the forward Euler method, the derivative is numerically approximated by forward finite differencing.

$$\frac{dy(x)}{dx} = f(x, y); \quad y = y_0 \text{ at } x = x_0$$
$$\frac{y(x + \Delta x) - y(x)}{\Delta x} = f(x, y)$$



- The implicit dependence of x on y is implied and not generally indicated by $y(x)$.
- Forward Euler is a first order method. i.e., $\text{error} = O(\Delta x)$.

Forward Euler Method (contd.)

- By taking small increments of Δx , forward marching can be employed to evaluate the value of y at any forward point x starting from x_0 .

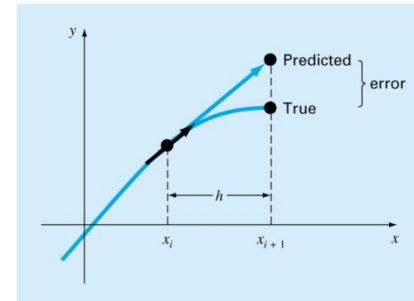
$$\frac{dy}{dx} = f(x, y); \quad y = y_0 \text{ at } x = x_0$$

$$\frac{y_{i+1} - y_i}{\Delta x} = f(x_i, y_i); \quad i = 0, 1, \dots, n \text{ and } x_i = x_{i-1} + \Delta x$$

$$y_{i+1} = y_i + f(x_i, y_i)\Delta x; \quad i = 0, 1, \dots, n \text{ and } x_i = x_{i-1} + \Delta x$$

- Or in compact notation:

$$y_{i+1} = y_i + f_i \Delta x$$



Numerical Solution of Parachute Problem using the Forward Euler Method

- Forward Euler can be applied as below for this problem:

$$\frac{dv}{dt} = g - \frac{c}{m}v$$
$$\frac{v(t + \Delta t) - v(t)}{\Delta t} = g - \frac{c}{m}v(t)$$

$$\boxed{v(t + \Delta t) = v(t) + \Delta t \left(g - \frac{c}{m}v(t) \right)}$$

Heun's Method

- Forward Euler can be improved by a predictor-corrector approach called Heun's method.
- The basic principle is to use an intermediate value of x and y instead of the old value.

$$y_{i+1} = y_i + f_{i+\frac{1}{2}} \Delta x$$

- Heun's method is a second order method. i.e., error = $O(\Delta x^2)$.

Single-Step Heun's Method

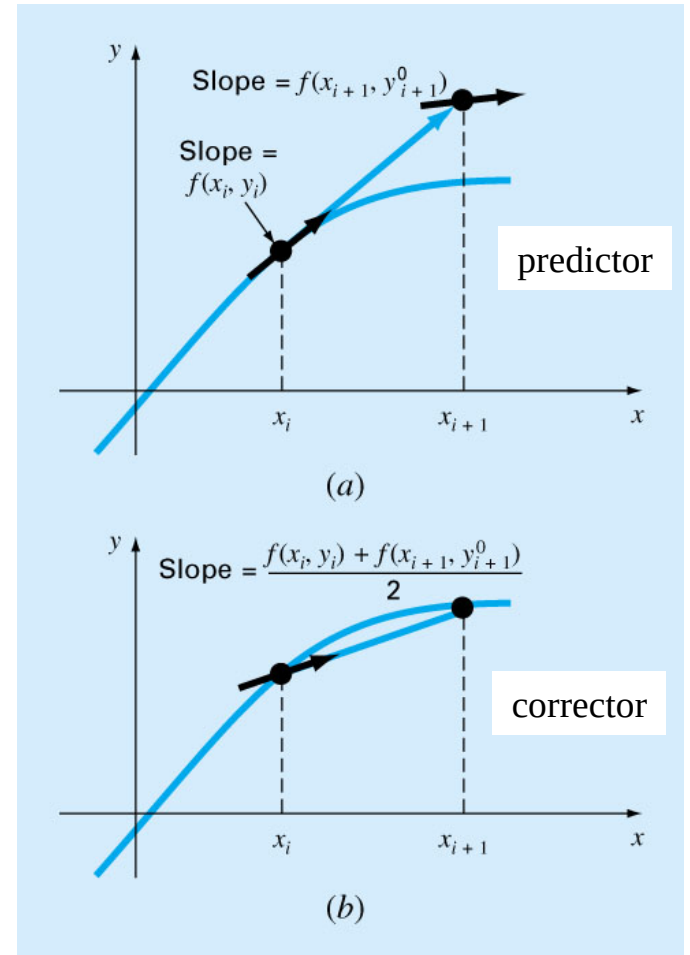
- Single step Heun's method involves 1 predictor and 1 corrector equation.

Predictor (Euler):

$$y_{i+1}^0 = y_i + f(x_i, y_i)\Delta x$$

Corrector :

$$y_{i+1} = y_i + \frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^0)}{2} \Delta x$$



Multi-Step Heun's Method

- Multi-step Heun's method: The corrector equation can be applied in an iterative fashion to produce better estimates of y_{i+1} . i.e.,

$$y_{i+1}^{j+1} = y_i + \frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^j)}{2} \Delta x$$

Mid-point method

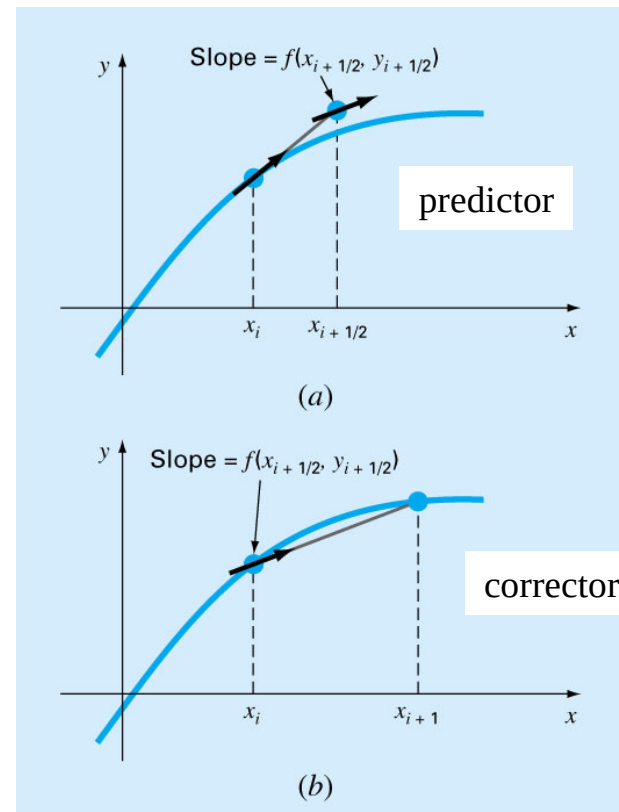
- Mid-point method is another predictor corrector modification of Euler's method that uses central differencing:

Predictor (Euler):

$$y_{i+1/2} = y_i + f(x_i, y_i) \frac{\Delta x}{2}$$

Corrector :

$$y_{i+1} = y_i + f(x_{i+1/2}, y_{i+1/2}) \Delta x$$



Runge-Kutta Methods

- Runge-Kutta methods attain higher order accuracy without calculation of higher order derivatives.
- The general form can be expressed as:

$$y_{i+1} = y_i + \phi(x_i, y_i, h)h$$

where ϕ is an increment function given by

$$\phi = a_1 k_1 + a_2 k_2 + \cdots + a_n k_n$$

where a 's are constants and k 's are given by

$$k_1 = f(x_i, y_i)$$

$$k_2 = f(x_i + p_1 h, y_i + q_{11} k_1 h)$$

$$k_3 = f(x_i + p_2 h, y_i + q_{21} k_1 h + q_{22} k_2 h)$$

$\vdots$

$$k_n = f(x_i + p_{n-1} h, y_i + q_{n-1,1} k_1 h + q_{n-1,2} k_2 h + \cdots + q_{n-1,n-1} k_{n-1} h)$$

where p 's and q 's are constants.

Runge-Kutta Methods

- Various RK methods can be devised for different values of n .
- Once n is chosen, the constants a 's, p 's and q 's are evaluated by setting the RK equation to a Taylor series expansion:

$$y_{i+1} = y_i + f(x_i, y_i)h + \frac{f'(x_i, y_i)}{2!}h^2 + \frac{f''(x_i, y_i)}{3!}h^3 + \dots$$

- For $n = 1$, RK method reduces to the Euler's method.
- For $n = 2$, various RK methods exist depending on the value of a_2 chosen (p. 494-495 SC, p.702-704 C&C).

Classical 4th Order Runge-Kutta

- This is the most popular RK method.
- It can be expressed by the following 5 simple relationships.

$$y_{i+1} = y_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

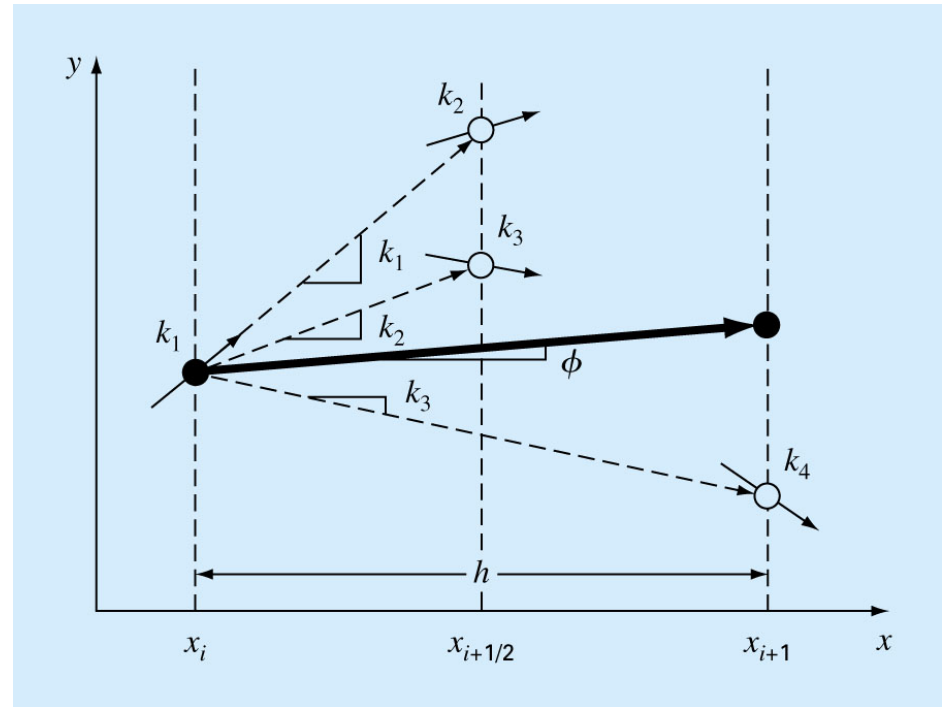
where

$$k_1 = f(x_i, y_i)$$

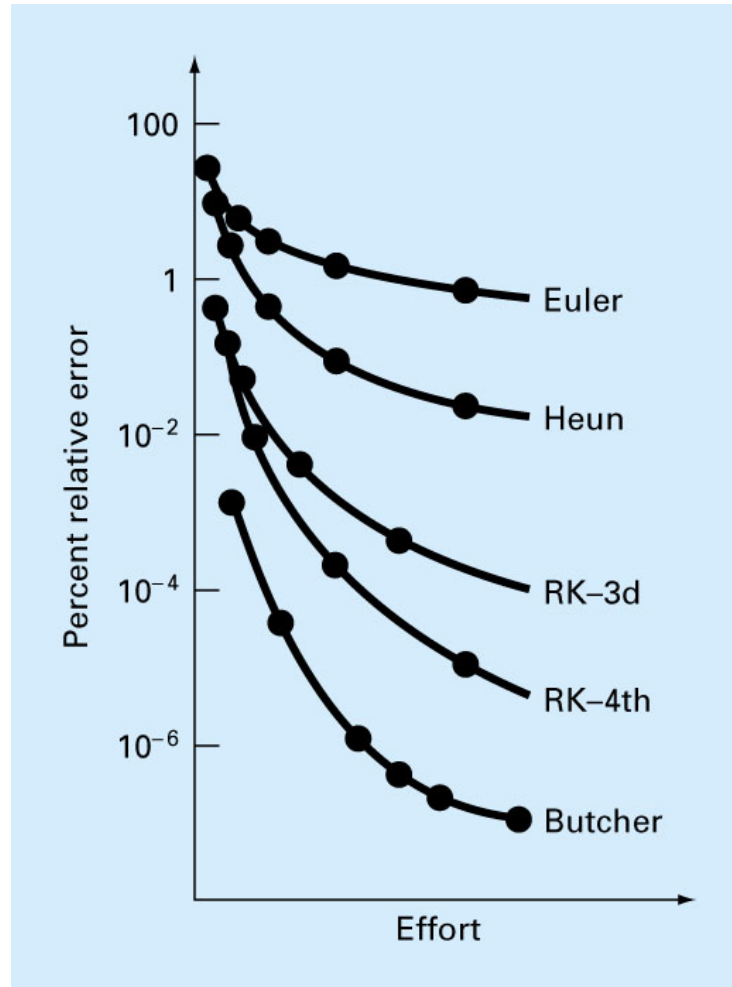
$$k_2 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1h\right)$$

$$k_3 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2h\right)$$

$$k_4 = f(x_i + h, y_i + k_3h)$$



Comparison of methods



In class Example (C&C p. 705)

- Solve the following ODE for y using Euler, Heuns, mid-point, and 4th order Runge-kutta for $x=0$ to 4

$$\frac{dy}{dx} = -2x^3 + 12x^2 - 20x + 8.5$$

Initial condition: $x = 0, y = 1$

- Compare your results with ODE45 and analytical

IVP ODE examples

- A simple function example (1st order single)
- Falling parachutist problem (1st order single)
- Pendulum problem (2nd order single)
- Chemical reactor problem (1st order system)

MATLAB Implementation

- Non-stiff IVP ODEs can be numerically solved using MATLAB's ode function ode45.
 - Ode45 uses adaptive 4th order RK method
- The symbolic function `dsolve` can also be used to solve the ODE analytically.
- We can also implement our own Euler, Heun's and RK4 functions.
 - [ode_examples.m](#)

A simple ODE example (C&C p. 705)

- 1st order ODE

$$\frac{dy}{dx} = -2x^3 + 12x^2 - 20x + 8.5$$

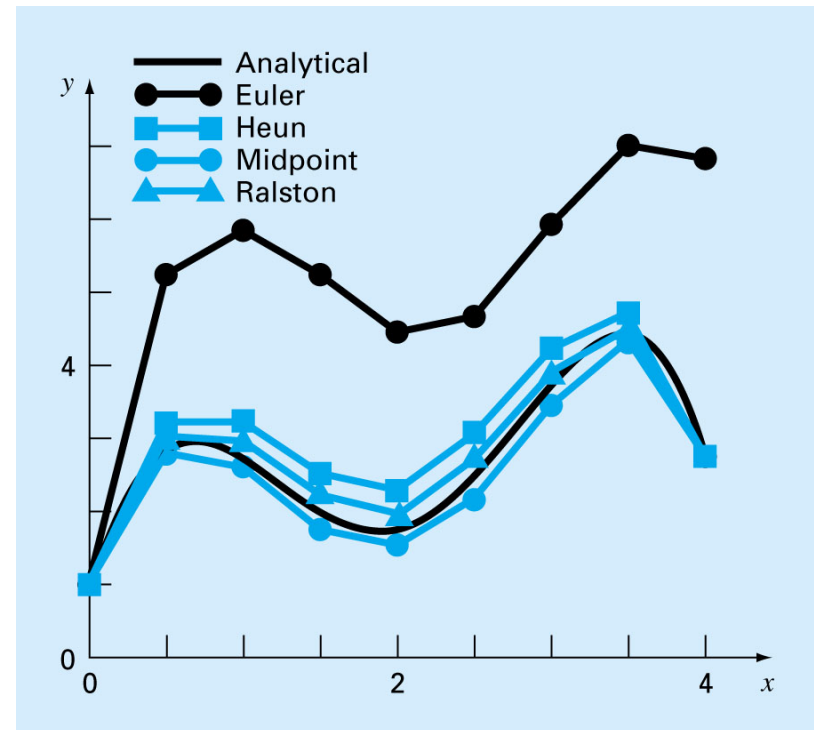
- Initial condition $x = 0, \quad y = 1$

- Analytical solution by integration

$$y = -0.5x^4 + 4x^3 - 10x^2 + 8.5x + 1$$

MATLAB implementation

- Write a MATLAB function to solve this problem for $x = 0$ to 4 with an interval of 0.1.
- Implement forward Euler, single-step Heuns, mid-point, and 4th order Runge-kutta functions.
- Compare the solutions in a plot of x vs y with Matlab's ode45 function and analytical solutions.
- Solution:
 - [simple ode example.m](#)



Another 1st order ODE Example

Parachute Problem

- Newton's second law ($f = ma$) can be applied to the falling parachutist problem.
- Two forces act, upward drag force f_u , and downward gravity force f_d

$$\downarrow f_d - f_u = ma$$

$$mg - cv = m \frac{dv}{dt}$$

$$\frac{dv}{dt} = g - \frac{c}{m}v$$



Analytical Solution

- The ODE can be solved analytically for the velocity $v(t)$ by separating the variables and then simply integrating.

$$\int \frac{dv}{g - \frac{c}{m}v} = \int dt$$

$$-\frac{m}{c} \ln \left(g - \frac{c}{m}v \right) = t + \text{const}$$

Analytical Solution (contd.)

- If the initial condition is $v = 0$ at $t = 0$,

$$const = -\frac{m}{c} \ln(g)$$

- Substituting we obtain

$$\begin{aligned} t &= -\frac{m}{c} \ln\left(g - \frac{c}{m}v\right) + \frac{m}{c} \ln(g) \\ &= -\frac{m}{c} \ln\left(\frac{g - \frac{c}{m}v}{g}\right) = -\frac{m}{c} \ln\left(1 - \frac{cv}{mg}\right) \end{aligned}$$

Falling Parachutist Problem (contd.)

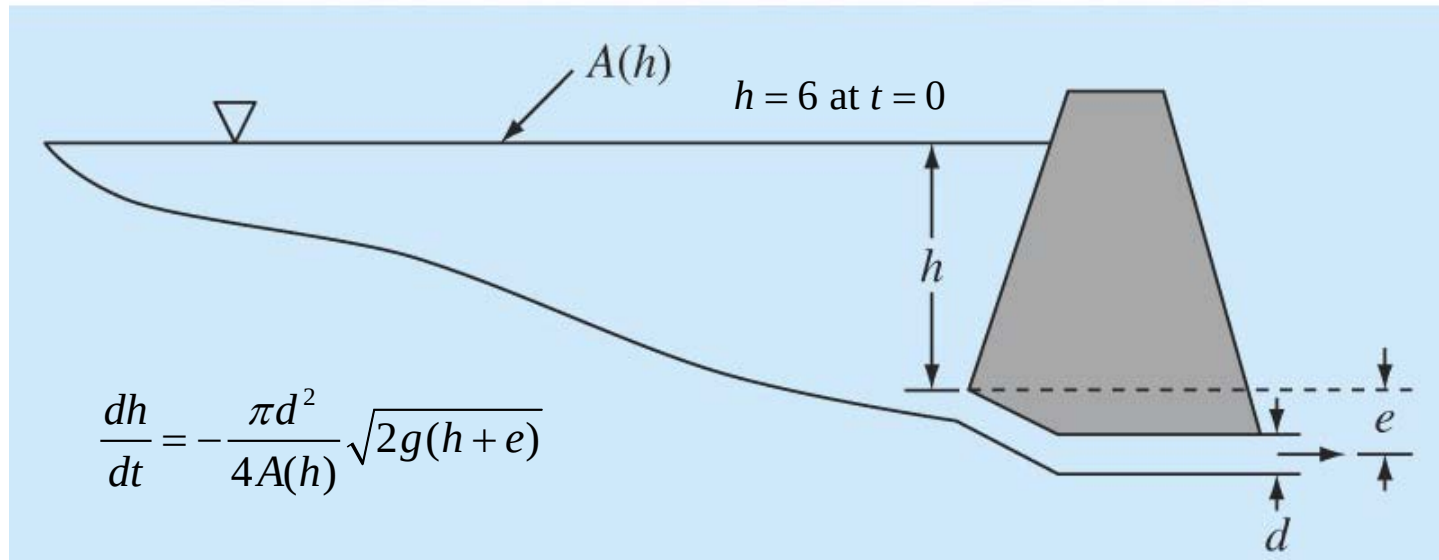
- Taking exponential on both sides,

$$e^{-\frac{ct}{m}} = 1 - \frac{cv}{gm}$$

- Therefore,

$$v(t) = \frac{gm}{c} \left(1 - e^{-\frac{ct}{m}} \right)$$

Reservoir draining problem (C & C p. 28.28, p. 833)



h (m)	6	5	4	3	2	1	0
A (ha)	1.17	0.97	0.67	0.45	0.32	0.18	0

Find time to drain

System of 1st order ODEs

- Solution to a system of n 1st order ODEs is possible if n initial conditions specified available at the starting value of x (i.e., x_0) as given below:

$$\frac{dy_1}{dx} = f_1(x, y_1, y_2, \dots, y_n) \quad x = x_0, y_1 = a_1$$

$$\frac{dy_2}{dx} = f_2(x, y_1, y_2, \dots, y_n) \quad x = x_0, y_2 = a_2$$

$\vdots$

$$\frac{dy_n}{dx} = f_n(x, y_1, y_2, \dots, y_n) \quad x = x_0, y_n = a_n$$

System of 1st order ODEs

- All the methods described earlier can be easily extended to a system of 1st order ODEs since they all work with known estimates of the dependent variables (explicit).
- For example, RK method can be written as:

$$y_{i+1}^j = y_i^j + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

where

$$k_1 = f(x_i, y_i^1, \dots, y_i^j, \dots, y_i^n)$$

$$k_2 = f(x_i + \frac{1}{2}h, y_i^1, \dots, y_i^j + \frac{1}{2}h, \dots, y_i^n)$$

$$k_3 = f(x_i + \frac{1}{2}h, y_i^1, \dots, y_i^j + \frac{1}{2}k_2h, \dots, y_i^n)$$

$$k_n = f(x_i + h, y_i^1, \dots, y_i^j + k_3h, \dots, y_i^n)$$

MATLAB implementation for systems of ODEs

- Our single ODE implementation can be easily extended to a system of ODEs by changing the dependent variable vector to a matrix containing multiple dependent variables.
- At each function evaluation a column of this matrix is passed in to the function corresponding to the desired point of evaluation.
- All methods can be included in a single function file that can be called with the desired method:
 - [ode_solver.m](#)

Systems of ODEs example

- Two 1st order ODEs

$$\frac{dy_1}{dx} = -0.5y_1; \quad \frac{dy_2}{dx} = 4 - 0.3y_2 - 0.1y_1$$

- Initial conditions

$$\text{At } x = 0, \quad y_1 = 4 \quad \text{and} \quad y_2 = 6$$

- Analytical solution:

$$y_1 = 4e^{-0.5x}; \quad y_2 = 2e^{-0.5x} - 9.33e^{-0.3x} + 13.33$$

MATLAB solution of ODE systems

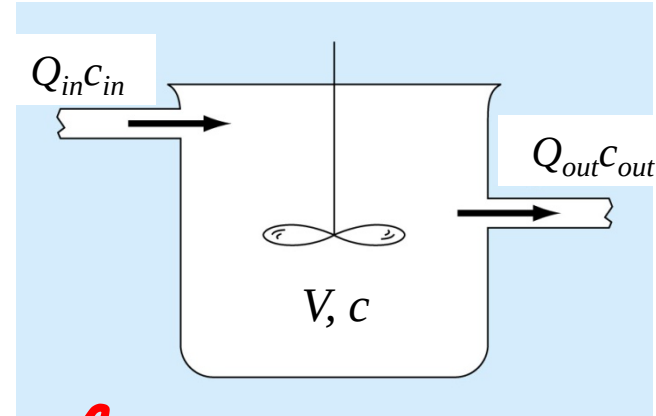
- A general MATLAB implementation for systems of ODEs can be used to solve the previous problem:
 - [simple_ode_system_example.m](#)

Chemical Reactor Example

- Mass balance Equation:

accumulation (mg/min) = input - output

$$V \frac{dc}{dt} = \sum Q_{in} c_{in} - \sum Q_{out} c_{out}$$



- Where

- V = volume of reactor (m^3)
- c = concentration in reactor
- c_{out} = output concentration (mg/m^3) = c , if completely mixed.
- c_{in} = input concentration (mg/m^3)
- Q = inlet flow rate (m^3/min)

$$\frac{dm}{dt} = \sum \dot{m}_{in} - \sum \dot{m}_{out}$$

$$V_i \frac{dc_i}{dt} = Q_{0i} c_{0i} + \sum_{\substack{j=1 \\ i \neq j}}^n Q_{ji} c_j - \sum_{j=1}^n Q_{ij} c_i$$

$$\dot{m} = QC$$

Five Reactor Example

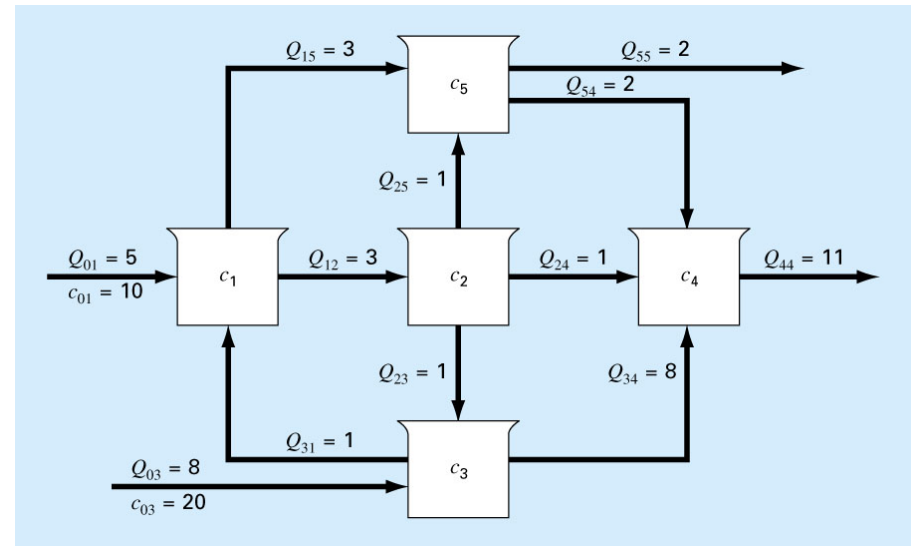
$$V_1 \frac{dc_1}{dt} = (Q_{01}c_{01} + Q_{31}c_3) - (Q_{12}c_1 + Q_{15}c_1)$$

$$V_2 \frac{dc_2}{dt} = (Q_{12}c_1) - (Q_{23}c_2 + Q_{24}c_2 + Q_{25}c_2)$$

$$V_3 \frac{dc_3}{dt} = (Q_{03}c_{03} + Q_{23}c_2) - (Q_{31}c_3 + Q_{34}c_3)$$

$$V_4 \frac{dc_4}{dt} = (Q_{24}c_2 + Q_{34}c_3 + Q_{54}c_5) - (Q_{44}c_4)$$

$$V_5 \frac{dc_5}{dt} = (Q_{15}c_1 + Q_{25}c_2) - (Q_{54}c_5 + Q_{55}c_5)$$



Initial condition:
at $t=0$; $c_1 = c_2 = c_3 = c_4 = c_5 = 0$

$$V_1 = 50 \text{ m}^3, V_2 = 20 \text{ m}^3, \\ V_3 = 40 \text{ m}^3, V_4 = 80 \text{ m}^3, V_5 = 100 \text{ m}^3$$

Five Reactor Example (contd.)

- The resulting ODE system is:

$$\frac{dc_1}{dt} = -0.12c_1 + 0.02c_3 + 1$$

$$\frac{dc_2}{dt} = 0.15c_1 - 0.15c_2$$

$$\frac{dc_3}{dt} = 0.025c_2 - 0.225c_3 + 4$$

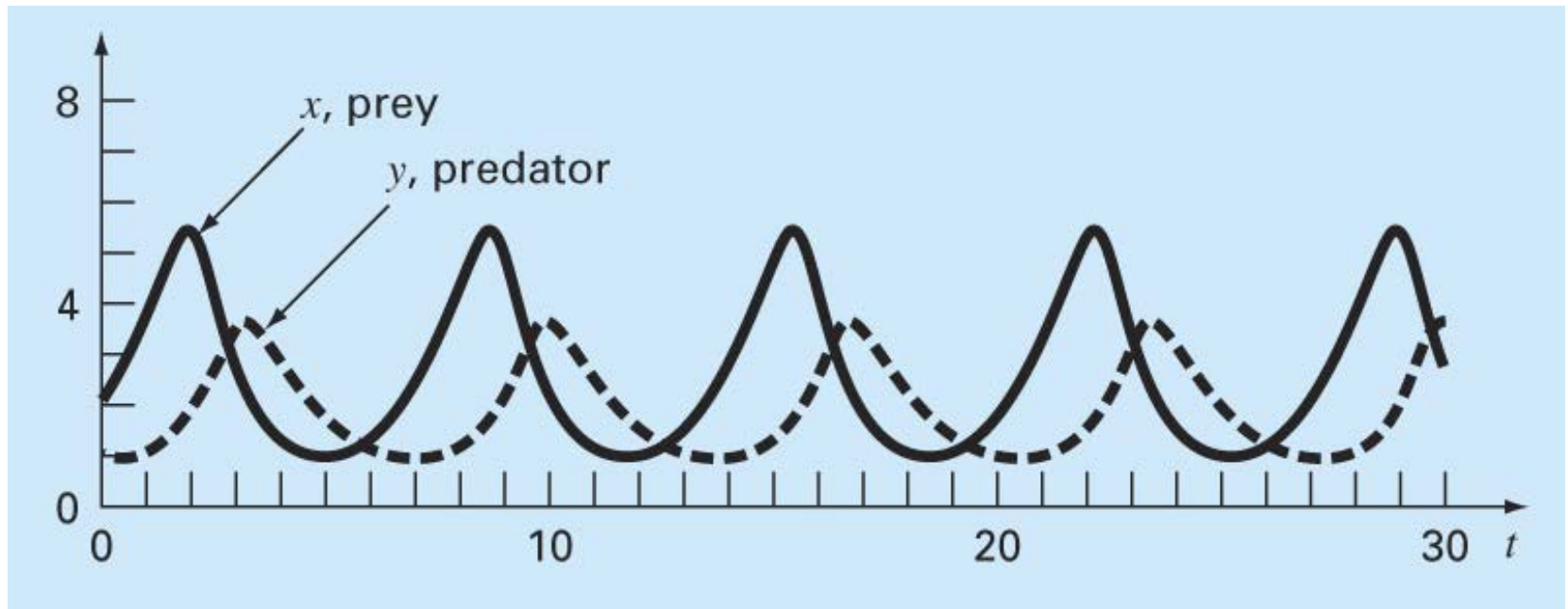
$$\frac{dc_4}{dt} = 0.1c_3 - 0.1375c_4 + 0.025c_5$$

$$\frac{dc_5}{dt} = 0.03c_1 + 0.01c_2 - 0.04c_5$$

Chemical Reactor - MATLAB Implementation

- This can be solved by three approaches.
 - Our ode solvers (euler, heuns, mid point, runge-kutta)
 - MATLAB's ode45 function
 - Analytical solution using symbolic 'dsolve'
- See:
 - [chemical_reactor_example.m](#)

Predator Prey Example (C&C p. 818-819)



Higher order ODEs

- Higher order ODEs can be solved by transforming them into a system of 1st order ODEs.
- For example, a 2nd order ODE can be transformed into a system of two 1st order ODEs.
- Higher order ODE example – pendulum problem

Higher Order ODEs – 2nd order example (CC 25.4 p 752)

- Solve the following ODE from $x = 0$ to 5 with $h = 0.5$

$$\frac{d^2 y}{dx^2} + 0.6 \frac{dy}{dx} + 8y = 0$$
$$y(0) = 2; y'(0) = 0$$

- Break into 2 first order ODEs

$$\text{Set } y_1 = y \text{ and } y_2 = \frac{dy}{dx}$$

$$\frac{dy_1}{dx} = y_2; \quad y_1 = 2, x = 0$$

$$\frac{dy_2}{dx} = -0.6y_2 + 8y_1; \quad y_2 = 0, x = 0$$

Pendulum Problem

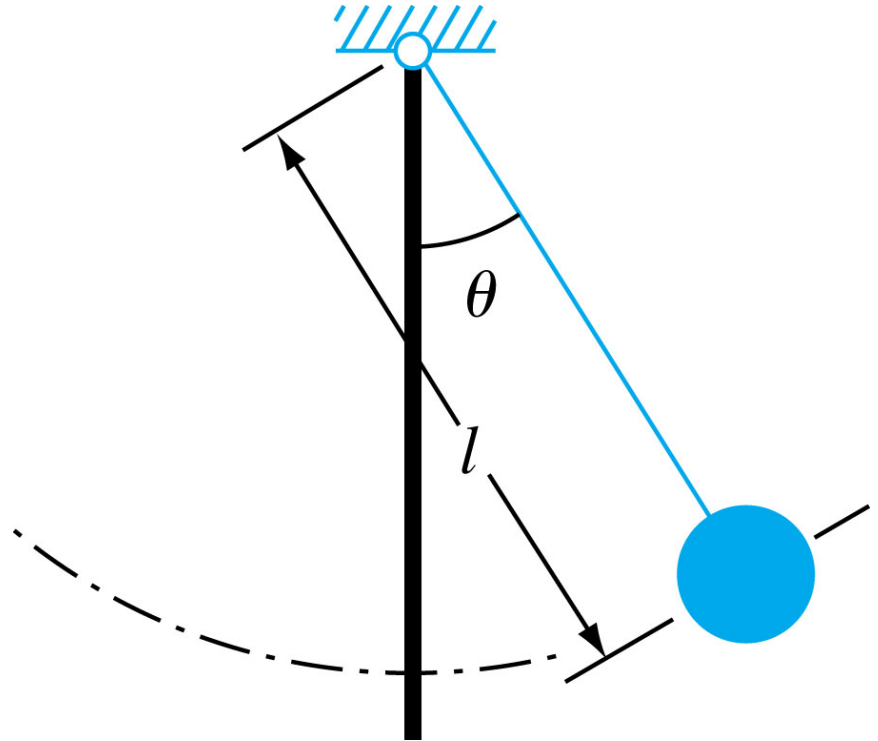
- Predict the motion of a swinging pendulum
- 2nd order ODE

$$\frac{d^2\theta}{dt^2} + \frac{g}{l}\sin\theta = 0$$

- Initial conditions:

$$\text{At } t = 0, \quad \theta = \pi / 4$$

$$\text{At } t = 0, \quad \frac{d\theta}{dt} = 0$$



Pendulum Problem (contd.)

- Transform 2nd order ODE into two 1st order ODEs by substituting $s = d\theta/dt$.

$$s = \frac{d\theta}{dt}; \quad t = 0, \quad \theta = \pi / 4$$

$$\frac{ds}{dt} + \frac{g}{l} \sin \theta = 0; \quad t = 0, \quad s = 0$$

- Or in a more convenient form ($\theta = y_1, s = y_2$).

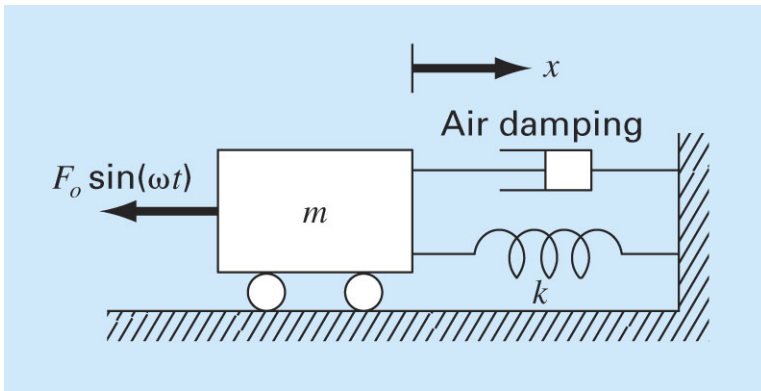
$$\frac{dy_1}{dt} = y_2; \quad t = 0, \quad y_1 = \pi / 4$$

$$\frac{dy_2}{dt} = -\frac{g}{l} \sin y_1; \quad t = 0, \quad y_2 = 0$$

Pendulum - MATLAB Implementation

- This can be solved by two approaches.
 - Our ode_solver function
 - MATLAB's ode45 function
- Analytical solution using symbolic 'dsolve' is not possible since closed form solutions are difficult to obtain.
- See:
 - [pendulum_example.m](#)

Forced Damping System Example (C & C 28.47, p. 836)



$$m \frac{d^2 x}{dt^2} + a \left| \frac{dx}{dt} \right| \frac{dx}{dt} + kx = F_0 \sin(\omega t)$$

$$t = 0; \quad \frac{dx}{dt} = 0$$

$$t = 0; \quad x = 1$$

$T = \text{time}$, $m = 2 \text{ kg mass}$, $a = 5 \text{ N}/(\text{m/s})^2$, $k = 6 \text{ N/m}$
 $F_0 = 2.5 \text{ N}$ and $\omega = 0.5 \text{ rad/s}$

Forced damping example

Stiff Systems

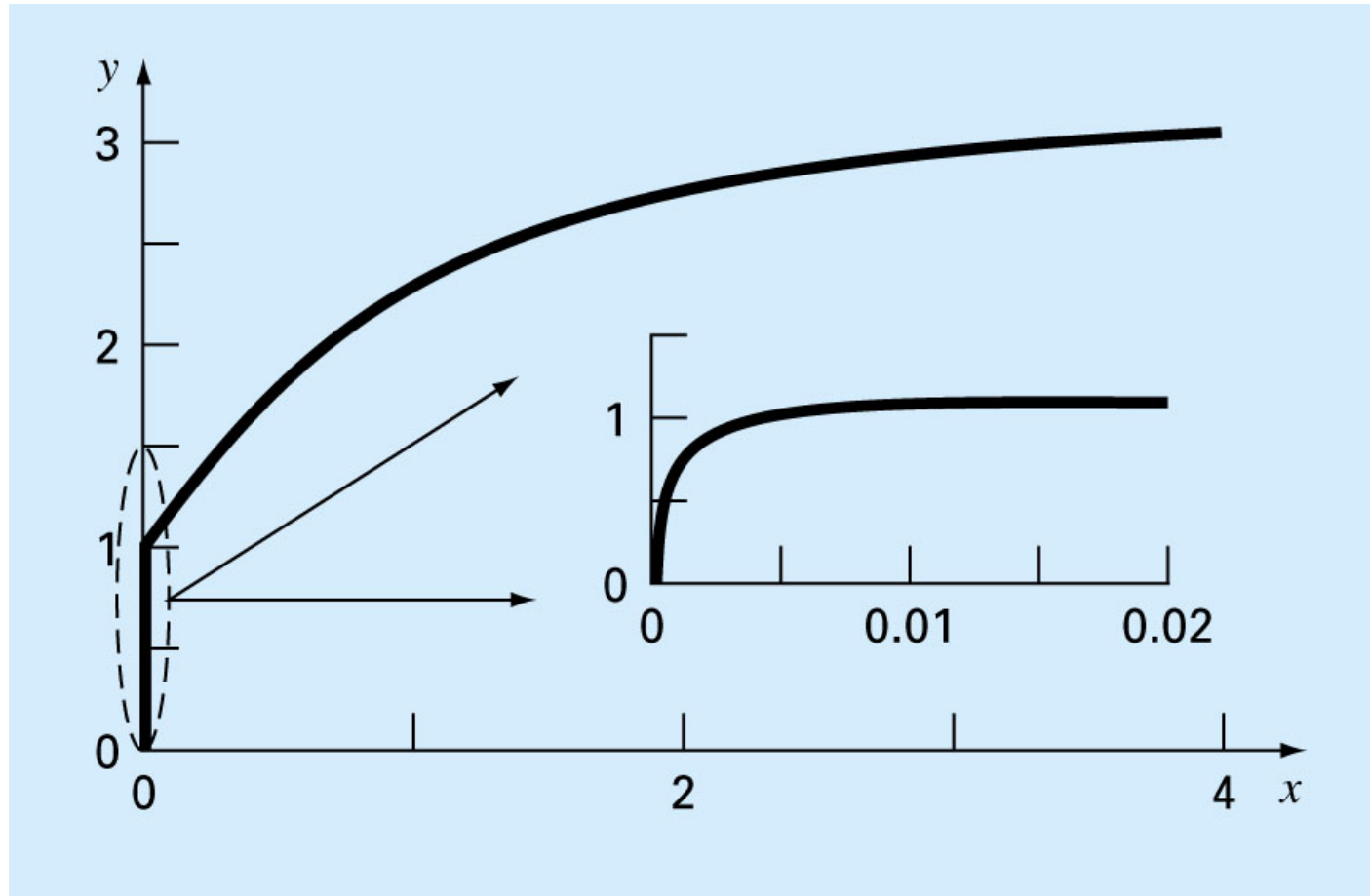
- What is a stiff ODE system ?
 - An ODE system that involves both rapidly changing and slowly changing components
- Example:

$$\frac{dy}{dt} = -1000y + 3000 - 2000e^{-t}$$

- With initial condition $y = 0$ at $t = 0$ the analytical solution is

$$y = 3 - 0.998e^{-1000t} - 2.002e^{-t}$$

Solution behavior of Stiff Example



Solution to Stiff Systems

- Explicit methods described earlier require very small step sizes to solve systems (hence a large number of function evaluations).
- Multi-step methods or Implicit or semi-implicit methods are more suited for stiff systems.
- A common implicit technique is the backward Euler.
- For nonlinear coupled stiff ODEs semi-implicit techniques are more commonly used due to the complexity of fully implicit methods.

MATLAB implementation for Stiff ODEs

- Backward Euler for linear ODE systems – requires solution of linear algebraic system.
- Multi-step Heuns method for linear and non-linear systems.
- Semi-implicit extrapolation method for non-linear systems – requires computation of Jacobian.
- MATLAB functions `ode15s`, `ode23s`.

Backward Euler for Stiff Example

[stiff_example.m](#)

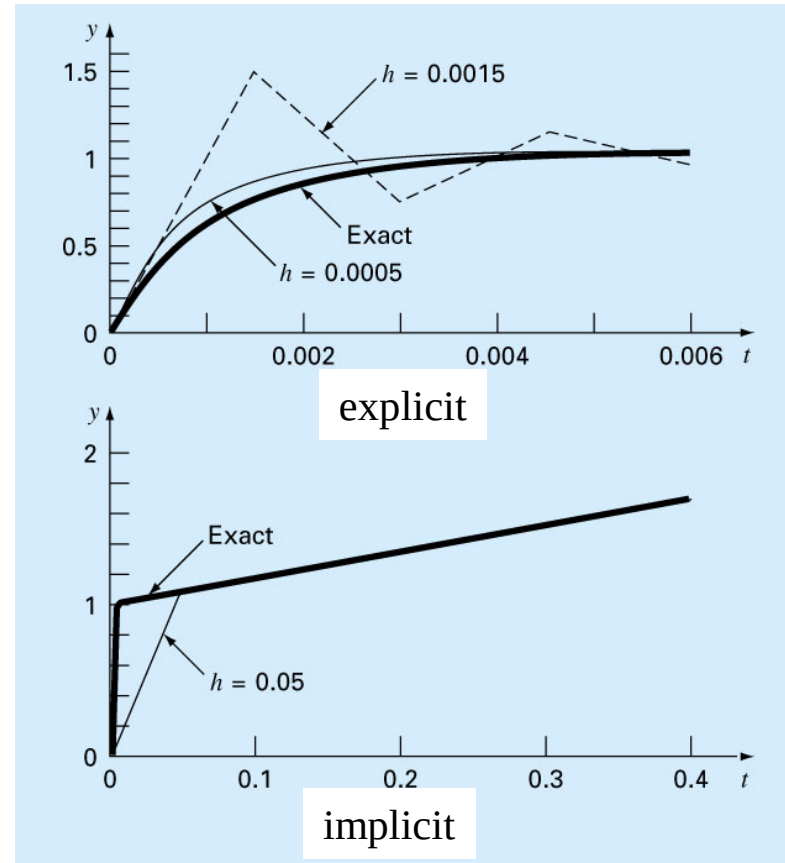
$$\frac{dy}{dt} = -1000y + 3000 - 2000e^{-t}$$

$$\frac{y^{k+1} - y^k}{\Delta t} = -1000y^{k+1} + 3000 - 2000e^{-t^{k+1}}$$

$$y^{k+1} = \frac{y^k + 3000\Delta t - 2000\Delta te^{-t^{k+1}}}{1 + 1000\Delta t}$$

Numerical Solution of Stiff Example

- Explicit Euler does not perform well for larger step sizes.
- Implicit Euler performs reasonably well even for larger step sizes.



Implicit method for a *system of stiff* ODEs

- If the system is linear with respect to the dependent variables then implicit method requires solution of a linear system.

- Example:

$$\frac{d\mathbf{y}}{dt} = \mathbf{A}\mathbf{y} + \mathbf{c}$$

- Where $\mathbf{A}$ is an $n \times n$ matrix, $\mathbf{y}$ is the $n \times 1$ unknown vector (dependent variable) and $\mathbf{c}$ is a known constant $n \times 1$ vector.
- Coefficients of $\mathbf{A}$ can be either constants or functions of independent variable t .

Fully Implicit method for a simple *linear system*

$$\frac{d\mathbf{y}}{dt} = \mathbf{A}\mathbf{y}$$

$$\frac{\mathbf{y}^{k+1} - \mathbf{y}^k}{\Delta t} = \mathbf{A}\mathbf{y}^{k+1}$$

$$\mathbf{y}^{k+1} - \mathbf{y}^k = \Delta t \mathbf{A} \mathbf{y}^{k+1}$$

$$\mathbf{y}^{k+1} (\mathbf{I} - \Delta t \mathbf{A}) = \mathbf{y}^k$$

$$\mathbf{y}^{k+1} = (\mathbf{I} - \Delta t \mathbf{A})^{-1} \mathbf{y}^k$$

identity matrix

Fully Implicit method for a general linear system

$$\frac{d\mathbf{y}}{dt} = \mathbf{A}(t)\mathbf{y} + \mathbf{c}(t)$$

$$\frac{\mathbf{y}^{k+1} - \mathbf{y}^k}{\Delta t} = \mathbf{A}(t^{k+1})\mathbf{y}^{k+1} + \mathbf{c}(t^{k+1}) \quad \text{set } \Delta\mathbf{y}^k = \mathbf{y}^{k+1} - \mathbf{y}^k$$

$$\Delta\mathbf{y}^k = \Delta t \mathbf{A}(t^{k+1}) (\mathbf{y}^k + \Delta\mathbf{y}^k) + \Delta t \mathbf{c}(t^{k+1})$$

$$\Delta\mathbf{y}^k (I - \Delta t \mathbf{A}(t^{k+1})) = \Delta t \mathbf{A}(t^{k+1}) \mathbf{y}^k + \Delta t \mathbf{c}(t^{k+1})$$

$$\Delta\mathbf{y}^k = (I - \Delta t \mathbf{A}(t^{k+1}))^{-1} \Delta t (\mathbf{A}(t^{k+1}) \mathbf{y}^k + \mathbf{c}(t^{k+1}))$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \Delta\mathbf{y}^k$$

Stiff System Example

- Stiff ODE system:

$$\frac{dy_1}{dt} = -5y_1 + 3y_2; \quad \frac{dy_2}{dt} = 100y_1 - 301y_2$$

- Initial conditions:

$$\text{At } t = 0, \quad y_1 = 52.29 \quad \text{and} \quad y_2 = 83.82$$

- MATLAB solution using fully implicit:

- [stiff_system_example.m](#)

$$\underline{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

$$A = \begin{bmatrix} -5 & 3 \\ 100 & -301 \end{bmatrix}$$

Semi-Implicit method for a non-linear system

$$\frac{d\mathbf{y}}{dt} = \mathbf{F}(\mathbf{y}, t)$$

$$\frac{\mathbf{y}^{k+1} - \mathbf{y}^k}{\Delta t} = \mathbf{F}(\mathbf{y}^{k+1}, t^{k+1}) \quad \text{set } \Delta\mathbf{y}^k = \mathbf{y}^{k+1} - \mathbf{y}^k$$

$$\frac{\Delta\mathbf{y}^k}{\Delta t} = \mathbf{F}(\mathbf{y}^k + \Delta\mathbf{y}^k, t^{k+1}) = \mathbf{F}(\mathbf{y}^k, t^{k+1}) + \Delta\mathbf{y}^k \frac{\partial \mathbf{F}(\mathbf{y}^k, t^{k+1})}{\partial \mathbf{y}} + O(\Delta\mathbf{y}^2)$$

$$\Delta\mathbf{y}^k \left(I - \Delta t \mathbf{J}(\mathbf{y}^k, t^{k+1}) \right) = \Delta t \mathbf{F}(\mathbf{y}^k, t^{k+1})$$

$$\Delta\mathbf{y}^k = \left(I - \Delta t \mathbf{J}(\mathbf{y}^k, t^{k+1}) \right)^{-1} \Delta t \mathbf{F}(\mathbf{y}^k, t^{k+1})$$

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \Delta\mathbf{y}^k$$

Semi-implicit ODE solver implementation for stiff systems

- Semi-implicit implementation (Newton's vector method) for a general system of stiff ODEs of the form $\frac{d\mathbf{y}}{dt} = \mathbf{F}(\mathbf{y}, t)$ can be implemented as follows using the previously derived expressions:

$$\Delta \mathbf{y}^k = \left(I - \Delta t \mathbf{J}(\mathbf{y}^k, t^{k+1}) \right)^{-1} \Delta t \mathbf{F}(\mathbf{y}^k, t^{k+1})$$
$$\mathbf{y}^{k+1} = \mathbf{y}^k + \Delta \mathbf{y}^k$$

- The Jacobian function derived earlier (see lecture notes for numerical differentiation) or analytical differentiation can be used to compute the Jacobian $\mathbf{J}$.

Nonlinear Stiff System Example

- Stiff ODE system:

$$\frac{dy_1}{dt} = -5y_1 + 3y_2;$$

$$\frac{dy_2}{dt} = 100y_1 - 301y_2^3$$

$$\underline{F} = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix} = \begin{bmatrix} -5y_1 + 3y_2 \\ 100y_1 - 301y_2^3 \end{bmatrix}$$

- Initial conditions:

$$\text{At } t = 0, \quad y_1 = 52.29 \quad \text{and} \quad y_2 = 83.82$$

- MATLAB solution using fully implicit:
 - [nonlinear stiff system example.m](#)

Analytical Jacobian for example problem

$$\begin{aligned} J &= \begin{bmatrix} \frac{\partial f_1}{\partial y_1} & \frac{\partial f_1}{\partial y_2} \\ \frac{\partial f_2}{\partial y_1} & \frac{\partial f_2}{\partial y_2} \end{bmatrix} \\ &= \begin{bmatrix} -5 & 3 \\ 100 & -903y_2^2 \end{bmatrix} \end{aligned}$$

Multi-Step Heun's Method

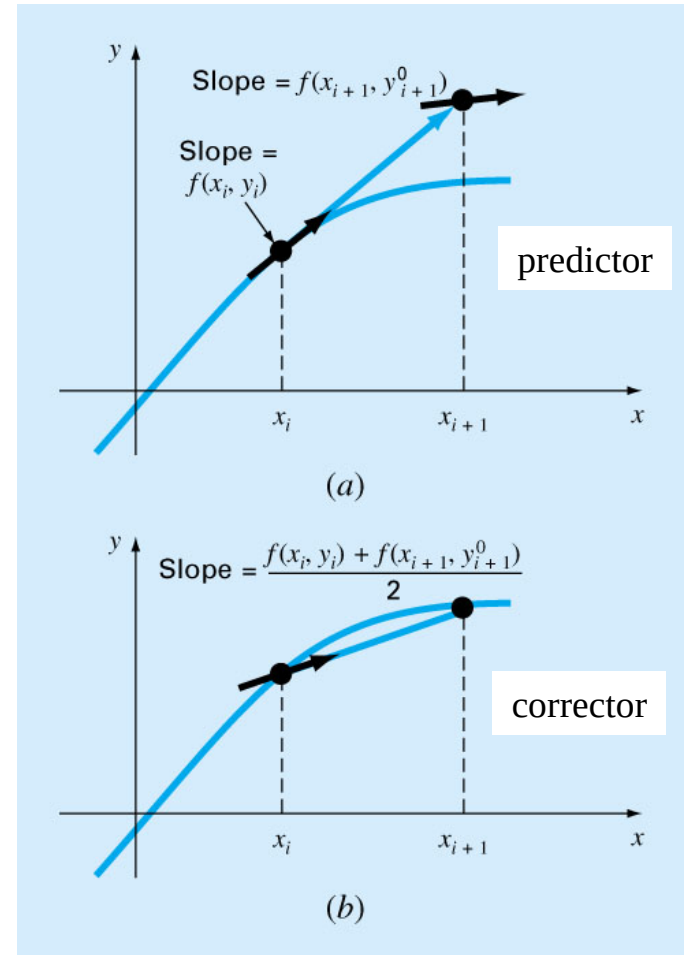
- In the multi-step Heun's method the corrector equation can be applied in an iterative fashion to produce better estimates of y_{i+1} .

Predictor (Euler):

$$y_{i+1}^0 = y_i + f(x_i, y_i)\Delta x$$

Corrector (for $j = 0, \dots, m$) :

$$y_{i+1}^{j+1} = y_i + \frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^j)}{2} \Delta x$$



Multi-Step Heun's Method

- Multi-step Heun's method: The corrector equation can be applied in an iterative fashion to produce better estimates of y_{i+1} . i.e.,

$$y_{i+1}^{j+1} = y_i + \frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^j)}{2} \Delta x$$

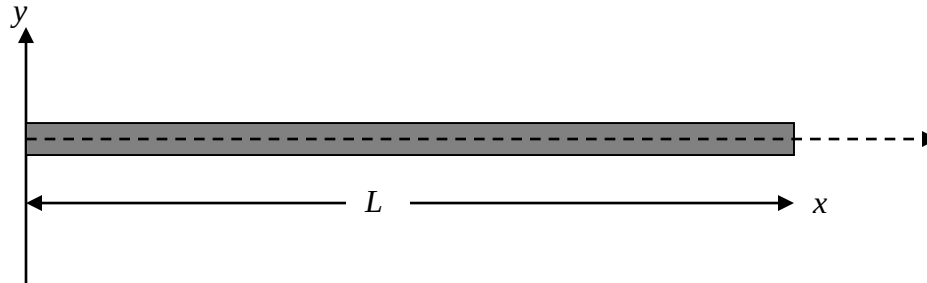
Boundary Value Problems (BVPs)

One-Point BVP ODEs

- One point BVP ODEs are similar to IVP ODEs
- Cantilever Example:

$$EI \frac{d^2 y}{dx^2} = -\rho g \left(1 + \left(\frac{dy}{dx} \right)^2 \right)^{3/2} \left[x \left(x - \frac{L}{2} \right) + \frac{L^2}{2} \right]$$

At $x = 0$, $y = 0$ and $\frac{dy}{dx} = 0$



Two-point BVP's

- Two-point boundary value problems require solution of an ODE similar to 1D PDEs
- Linear two-point BVP's are of the form

$$\frac{d^2 y}{dx^2} + p(x) \frac{dy}{dx} + q(x)y = g(x); \quad y(x_0) = y_0, \quad y(x_f) = y_f$$

- General two-point BVP's are of the form

$$\frac{d^2 y}{dx^2} = g\left(x, y, \frac{dy}{dx}\right); \quad y(x_0) = y_0, \quad y(x_f) = y_f$$

Solving Two-Point BVPs

- Finite-differences
 - Central FD
 - Boundary conditions
 - Matrix solution
 - Matlab function: `bvp4c`
- Other methods
 - Shooting Methods
 - Superposition method

Finite Difference Approximation (FDA) for BVP's

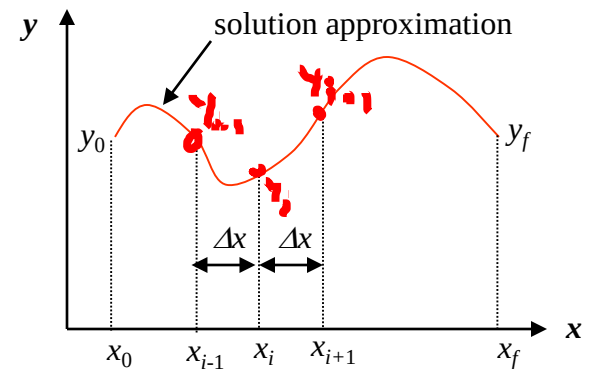
- Let's first assume the following linear BVP form

$$\frac{d^2 y}{dx^2} + p(x) \frac{dy}{dx} + q(x)y = g(x); \quad y(x_0) = y_0, \quad y(x_f) = y_f$$

- Central differences

$$\frac{dy}{dx} \approx \frac{y(x_{i+1}) - y(x_{i-1}))}{2(x_{i+1} - x_{i-1})} = \frac{y_{i+1} - y_{i-1}}{2\Delta x}$$

$$\frac{d^2 y}{dx^2} \approx \frac{y(x_{i+1}) - 2y(x_i) + y(x_{i-1}))}{\Delta x^2} = \frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2}$$



FDA for BVP's (contd.)

- Applying central differences to ODE gives us

$$\left(\frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2} \right) + p_i \left(\frac{y_{i+1} - y_{i-1}}{2\Delta x} \right) + q_i y_i = g_i, \quad i = 1, \dots, n$$

- Rearranging terms and multiplying by Δx^2

$$\left(1 - \frac{\Delta x}{2} p_i \right) y_{i-1} + (-2 + \Delta x^2 q_i) y_i + \left(1 + \frac{\Delta x}{2} p_i \right) y_{i+1} = \Delta x^2 g_i, \quad i = 1, \dots, n$$

FDA for BVP's (contd.)

- With $y_0 = y_0$ and $y_{n+1} = y_f$ we can get a system of equations of the form

$$\begin{pmatrix} -2 + \Delta x^2 q_1 & 1 + \frac{\Delta x}{2} p_1 & 0 & \cdots & 0 \\ 1 - \frac{\Delta x}{2} p_2 & -2 + \Delta x^2 q_2 & 1 + \frac{\Delta x}{2} p_2 & \cdots & 0 \\ 0 & 1 - \frac{\Delta x}{2} p_3 & -2 + \Delta x^2 q_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -2 + \Delta x^2 q_n \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} \Delta x^2 g_1 - \left(1 - \frac{\Delta x}{2} p_1\right) y_0 \\ \Delta x^2 g_2 \\ \Delta x^2 g_3 \\ \vdots \\ \Delta x^2 g_n - \left(1 + \frac{\Delta x}{2} p_n\right) y_{n+1} \end{pmatrix}$$

$$\mathbf{A} \mathbf{y} = \mathbf{b}$$

Solution to linear systems resulting from FDA of BVPs

- MATLAB's ' $\backslash$ ' operator can be used to solve the systems of linear equations resulting from FDA of BVP odes.
 - $y=A\backslash b$
- The ' $\backslash$ ' operator uses Gaussian elimination
 - May not be efficient for large systems
- Later on we will look at some iterative techniques for linear system solution
 - Jacobi
 - Gauss-Seidel
 - SOR
 - Conjugate gradients

MATLAB's BVP Function

- MATLAB function `bvp4c` can be used to solve linear and non-linear bvp's.
- `bvp4c` has the following requirements:
 - The 2nd order bvp ODE should be first transformed to a system of two first order odes (like in Runge-Kutta)
 - The two end boundary conditions should be given in a *vector function* of the form $y(x)=0$.
 - The initial condition has to be given as a structure (by calling the MATLAB's `bvpinit` function)
 - The solution needs to be extracted for desired values of x using the `deval` function.

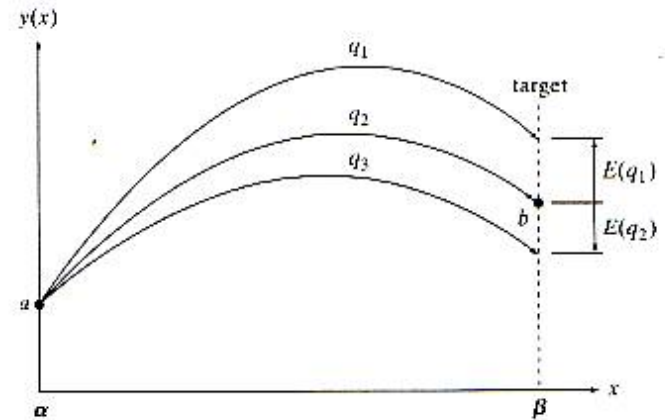
Shooting Method

- Shooting method is a method that first converts the BVP into an equivalent IVP and then uses a trial-and-error approach to solve the IVP.
- For linear problems the shooting method needs to solve the equivalent IVP system only twice as linear interpolation can be used.
- For nonlinear problems, many iterations may be needed or a root finding method may be used instead of interpolation.

Shooting Method (contd.)

- Original system

$$\frac{d^2 y}{dx^2} = g\left(x, y, \frac{dy}{dx}\right)$$
$$y(\alpha) = a, \quad y(\beta) = b$$



- Convert to two first order IVP ODEs

$$y_1 = y, \quad y_2 = \frac{dy}{dx} \Rightarrow$$

$$\frac{dy_1}{dx} = y_2; \quad \frac{dy_2}{dx} = g(x, y_1, y_2)$$

$$y_1(\alpha) = a, \quad y_1(\beta) = b, \quad y_2(\alpha) = ?$$

Our objective is to find $y_2(\alpha)$ such that $y_1(\beta) = b$.

Shooting Method Steps – Linear Problem

1. Convert the BVP ODE into two IVP ODEs.
2. Assume a value for dy/dx (say, q_1) at $x = \alpha$ (i.e., $y_2(\alpha)$) and solve this system using a standard method such as Runge-Kutta. Store the value of y at $x = \beta$ (i.e., $y_1(\beta) = p_1$).
3. Assume another value for dy/dx (say, q_2) at $x = \alpha$ (i.e., $y_2(\alpha)$) and solve this system using a standard method such as Runge-Kutta. Store the value of y at $x = \beta$ (i.e., $y_1(\beta) = p_2$).
4. Calculate the correct value of $y_2(\alpha)$ from these values by linear interpolation using the actual value of y at $x = \beta$ (i.e., $y_1(\beta) = b$).

$$y_2(\alpha) = q_1 + \frac{b - p_1}{p_2 - p_1}(q_2 - q_1)$$

5. Solve the new system using the same method (Runge-Kutta) with these initial conditions.

Shooting Method Steps – Non-Linear Problem

1. Convert the BVP ODE into two IVP ODEs.
2. Assume a value for $z_2(\alpha)$ and solve this system using a standard method such as Runge-Kutta.
3. Calculate the value of $z_1(\beta)$ from this solution and see if it matches b . If it matches terminate. If not, go to step 2 and iterate until it matches.
 - a root finding technique or a nonlinear least squares method can be used to guide the iteration (e.g., matlab function `fsolve`).

Linear two-point BVP examples

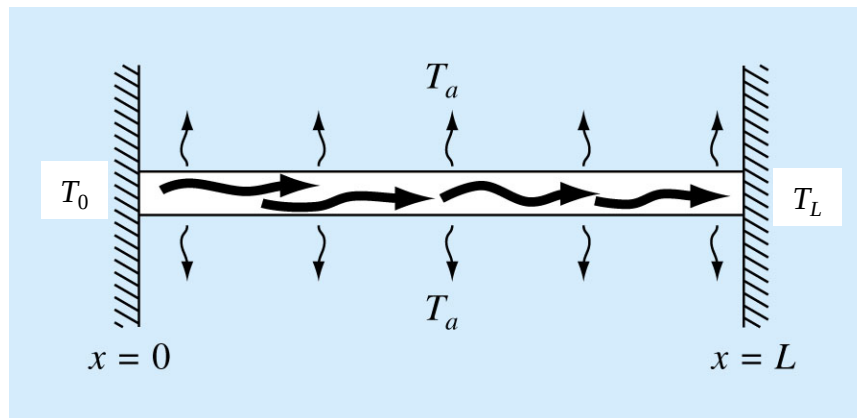
- Heat transfer
- Simply supported beam undergoing small deflections

Linear BVP – Heat Transfer (C&C p. 753)

- The ODE is given by:

$$\frac{d^2 T}{dx^2} + \alpha(T_a - T) = 0; \quad T(0) = T_0, \quad T(L) = T_L$$

$$L = 10, \quad \alpha = 0.01, \quad T_a = 20, \quad T_0 = 40, \quad T_L = 200$$



Heat Transfer Example (contd.)

- For matlab function `bvp4c` and shooting method transform using ($y_2 = dT/dx$, $y_1 = T$):

$$\frac{dy_1}{dx} = y_2$$

$$\frac{dy_2}{dx} = \alpha(y_1 - T_a)$$

$$y_1(0) = T_0, \quad y_1(L) = T_L$$

FDA for Heat Transfer Example

- Applying central differences to the 2nd order ODE gives us

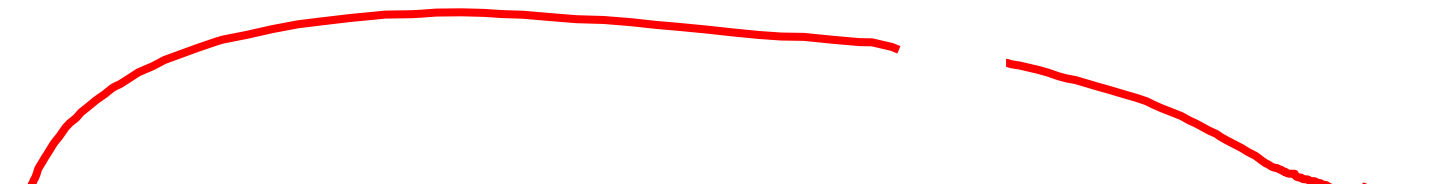
$$\left(\frac{T_{i+1} - 2T_i + T_{i-1}}{\Delta x^2} \right) - \alpha T_i = -\alpha T_a, \quad i = 1, \dots, n$$

- Rearranging terms and multiplying by Δx^2

$$T_{i-1} + (-2 - \Delta x^2 \alpha) T_i + T_{i+1} = -\Delta x^2 \alpha T_a, \quad i = 1, \dots, n$$

FDA for Heat Example (contd).

- With $T_0 = T_0$ and $T_{n+1} = T_L$ we can get a system of equations of the form



$$\begin{pmatrix} -2 - \Delta x^2 \alpha & 1 & 0 & \cdots & 0 \\ 1 & -2 - \Delta x^2 \alpha & 1 & \cdots & 0 \\ 0 & 1 & -2 - \Delta x^2 \alpha & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -2 - \Delta x^2 \alpha \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_n \end{pmatrix} = \begin{pmatrix} -\Delta x^2 \alpha T_a - T_0 \\ -\Delta x^2 \alpha T_a \\ -\Delta x^2 \alpha T_a \\ \vdots \\ -\Delta x^2 \alpha T_a - T_L \end{pmatrix}$$

$$\mathbf{A}\mathbf{y} = \mathbf{b}$$

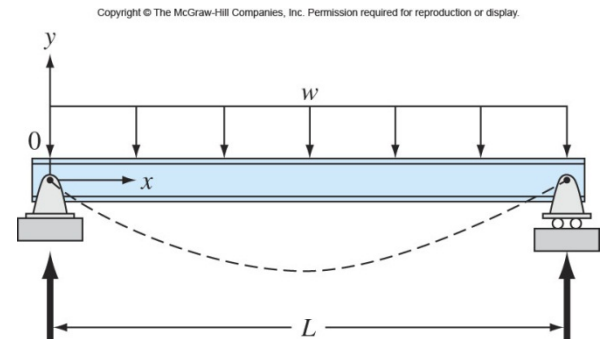
Linear BVP – Beam

- The ODE is given by:

$$EI \frac{d^2 y}{dx^2} = -\frac{w}{2}(x^2 - Lx); \quad y(0) = 0, \quad y(L) = 0$$

- For matlab function `bvp4c` and shooting method transform using ($y_2 = dy/dx$, $y_1 = y$):

$$\begin{aligned} \frac{dy_1}{dx} &= y_2 \\ \frac{dy_2}{dx} &= -\frac{w}{2EI}(x^2 - Lx) \\ y_1(0) &= 0, \quad y_1(L) = 0 \end{aligned}$$



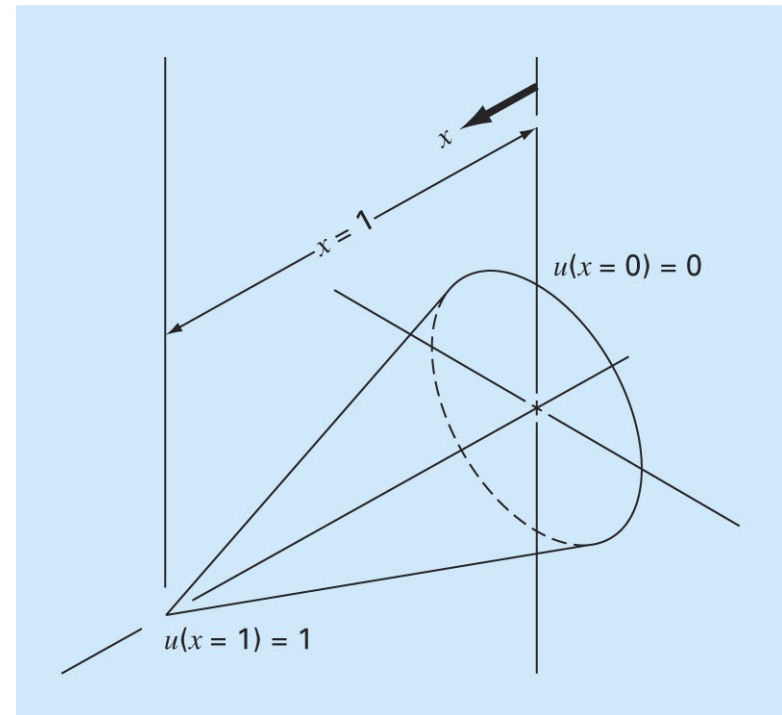
Heat transfer in cooling fin (C & C 28.48 p. 836)

- Non-dimensionalized heat equation in a conical cooling fin

$$\frac{d^2u}{dx^2} + \left(\frac{2}{x}\right)\left(\frac{du}{dx} - pu\right) = 0$$

- x = axial distance, u = temperature, p = nondimensional parameter
- Solve for $p = 10, 20, 50, 100$

Copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.



MATLAB code for linear BVPs

- Linear heat example is in:
 - [bvp_linear_heat_example.m](#)
- Beam example is not provided here as it is part of assignment 4.
- **Cooling fin example – do it as an exercise**

Non-linear BVPs

- An example of a non-linear BVP is

$$\frac{d^2 y}{dx^2} + p(x)y^2 \frac{dy}{dx} + q(x)y^2 = g(x); \quad y(x_0) = y_0, \quad y(x_f) = y_f$$

- Non-linear BVP's can be solved in the following ways
 - FDA with Explicit linearization
 - FDA with Newton's vector method
 - Shooting method with a root finding technique (already seen)
 - bvp4c

FDA with Explicit Linearization

- Example Non-linear ODE

$$\frac{d^2 y}{dx^2} + p(x) y^2 \frac{dy}{dx} + q(x) y^2 = g(x); \quad y(x_0) = y_0, \quad y(x_f) = y_f$$

- Linearize the ODE by simply lagging all non-linear coefficients

$$\left(\frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2} \right)^{(k+1)} + p_i \left(y_i^{(k)} \right)^2 \left(\frac{y_{i+1} - y_{i-1}}{2\Delta x} \right)^{(k+1)} + q_i y_i^{(k)} y_i^{(k+1)} = g_i, \quad i = 1, \dots, n$$

- Multiplying by Δx^2 and rearranging terms gives us

$$y_{i-1}^{(k+1)} \left(1 - \frac{\Delta x}{2} p_i \left(y_i^{(k)} \right)^2 \right) + y_i^{(k+1)} \left(-2 + \Delta x^2 q_i y_i^{(k)} \right) + y_{i+1}^{(k+1)} \left(1 + \frac{\Delta x}{2} p_i \left(y_i^{(k)} \right)^2 \right) = \Delta x^2 g_i, \quad i = 1, \dots, n$$

Newton's Vector Method

- Solve nonlinear system of equations of the form

$$f_i(\mathbf{y}) = 0 \quad 1 \leq i \leq n$$

- Compute $n \times n$ Jacobian matrix J given by

$$J_{ij} = \frac{\partial f_i(\mathbf{y})}{\partial y_j} \quad 1 \leq i, j \leq n$$

- Perform iteration

$$y_i^{k+1} = y_i^k - J^{-1}(y^k) f_i(y^k) \quad 1 \leq i \leq n, \quad k \geq 0$$

- More efficient to solve ($n^3/3$ ops) for correction term than inverting J ($4n^3/3$ ops)

$$\Delta y_i^k = y_i^{k+1} - y_i^k$$

$$J(y^k) \Delta y_i^k = -f_i(y^k) \quad 1 \leq i \leq n, \quad k \geq 0$$

$$y_i^{k+1} = y_i^k + \Delta y_i^k$$

FDA with Newton's Method for example ODE

- Example Non-linear ODE

$$\frac{d^2 y}{dx^2} + p(x)y^2 \frac{dy}{dx} + q(x)y^2 = g(x); \quad y(x_0) = y_0, \quad y(x_f) = y_f$$

- FDA yields

$$E_i(\mathbf{y}) = \left(1 - \frac{\Delta x}{2} p_i y_i^2\right) y_{i-1} + \left(-2 + \Delta x^2 q_i y_i^2\right) y_i + \left(1 + \frac{\Delta x}{2} p_i y_i^2\right) y_{i+1} - \Delta x^2 g_i$$
$$= 0, \quad i = 1, \dots, n$$

Newton's method Sample ODE (contd.)

- Newton's method

$$\mathbf{y}^{(k+1)} = \mathbf{y}^{(k)} - \left(\frac{\partial E(\mathbf{y}^{(k)})}{\partial \mathbf{y}} \right)^{-1} E(\mathbf{y}^{(k)})$$

- Rewrite equation to solve for residual $\Delta \mathbf{y}^{(k)} = \mathbf{y}^{(k+1)} - \mathbf{y}^{(k)}$

$$J(\mathbf{y}^{(k)}) \Delta \mathbf{y}^{(k)} = -E(\mathbf{y}^{(k)})$$

- Where the Jacobian J is given by

$$J(\mathbf{y}^{(k)}) = \left(\frac{\partial E(\mathbf{y}^{(k)})}{\partial \mathbf{y}} \right)$$

Basics Steps in Newton's (contd.)

- Compute Jacobian matrix J is given by

$$J(\mathbf{y}^{(k)}) = \left(\frac{\partial E(\mathbf{y}^{(k)})}{\partial \mathbf{y}} \right)$$

- In each step k solve for residual

$$J(\mathbf{y}^{(k)}) \Delta \mathbf{y}^{(k)} = -E(\mathbf{y}^{(k)})$$

- Update new value of vector $\mathbf{y}$

$$\mathbf{y}^{(k+1)} = \mathbf{y}^{(k)} + \Delta \mathbf{y}^{(k)}$$

Jacobian Matrix Computation

- The Jacobian matrix elements are given by

$$J_{ij} = \frac{\partial E_i}{\partial y_j}$$

$$E_i = \left(1 - \frac{\Delta x}{2} p_i y_i^2\right) y_{i-1} + \left(-2 + \Delta x^2 q_i y_i^2\right) y_i + \left(1 + \frac{\Delta x}{2} p_i y_i^2\right) y_{i+1} - \Delta x^2 g_i$$

- The i^{th} row of the $n \times n$ Jacobian matrix J is given by

$$J_{i,i-1} = \frac{\partial E_i}{\partial y_{i-1}} = 1 - \frac{\Delta x}{2} p_i \left(y_i^{(k)}\right)^2$$

$$J_{i,i} = \frac{\partial E_i}{\partial y_i} = -\Delta x p_i y_i^{(k)} y_{i-1}^{(k)} - 2 + 3\Delta x^2 q_i \left(y_i^{(k)}\right)^2 + \Delta x p_i y_i^{(k)} y_{i+1}^{(k)}$$

$$J_{i,i+1} = \frac{\partial E_i}{\partial y_{i+1}} = 1 + \frac{\Delta x}{2} p_i \left(y_i^{(k)}\right)^2$$

Jacobian Matrix Computation

- In the previous slide the Jacobian matrix was computed analytically.
- However, this is not always possible.
- In many cases, the Jacobian matrix may need to be computed numerically.
- When the Jacobian matrix is computed numerically using a first order approximation, the resulting Newton's vector method is also called the *secant method*.

General Secant Method

- Solve nonlinear system of equations of the form

$$f_i(y) = 0 \quad 1 \leq i \leq n$$

- Compute $n \times n$ Jacobian matrix $J(y)$ given by

$$J_{ij}(y) = \frac{\partial f_i(y)}{\partial y_j} = \frac{f_i(y_j + \delta y_j) - f_i(y_j)}{\delta y_j} \quad 1 \leq i, j \leq n$$

- Perform iteration

$$\Delta y_i^k = y_i^{k+1} - y_i^k$$

$$J(y^k) \Delta y_i^k = -f_i(y^k) \quad 1 \leq i \leq n, \quad k \geq 0$$

$$y_i^{k+1} = y_i^k + \Delta y_i^k$$

Secant Method applied to Example Problem

- The elements of the Jacobian matrix are given by

$$J_{ij} = \frac{\partial E_i}{\partial y_j} = \frac{E_i(y_j + \delta y_j) - E_i(y_j)}{\delta y_j}$$

- Where

$$E_i = \left(1 - \frac{\Delta x}{2} p_i y_i^2\right) y_{i-1} + \left(-2 + \Delta x^2 q_i y_i^2\right) y_i + \left(1 + \frac{\Delta x}{2} p_i y_i^2\right) y_{i+1} - \Delta x^2 g_i$$

- In general, a constant perturbation δy can be assumed for all values of i and j

$$J_{ij} = \frac{E_i(y_j + \delta y) - E_i(y)}{\delta y}$$

Secant Method for Example Problem (contd.)

- The i^{th} row of the $n \times n$ Jacobian matrix J is given by

$$J_{i,i-1} = \frac{1}{\delta y} \left\{ \left(1 - \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right) (y_{i-1}^{(k)} + \delta y) - \left(1 - \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right) \delta y \right\}$$

$$= \left(1 - \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right)$$

$$J_{i,i} = \frac{1}{\delta y} \left\{ \begin{aligned} & \left(1 - \frac{\Delta x}{2} p_i(y_i^{(k)} + \delta y)^2 \right) y_{i-1}^{(k)} - 2(y_i^{(k)} + \delta y) + \Delta x^2 q_i(y_i^{(k)} + \delta y)^3 + \left(1 + \frac{\Delta x}{2} p_i(y_i^{(k)} + \delta y)^2 \right) y_{i+1}^{(k)} \\ & - \left(1 - \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right) y_{i-1}^{(k)} - 2(y_i^{(k)}) + \Delta x^2 q_i(y_i^{(k)})^3 + \left(1 + \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right) y_{i+1}^{(k)} \end{aligned} \right\}$$

$$J_{i,i+1} = \frac{1}{\delta y} \left\{ \left(1 + \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right) (y_{i-1}^{(k)} + \delta y) - \left(1 + \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right) \delta y \right\}$$

$$= \left(1 + \frac{\Delta x}{2} p_i(y_i^{(k)})^2 \right)$$

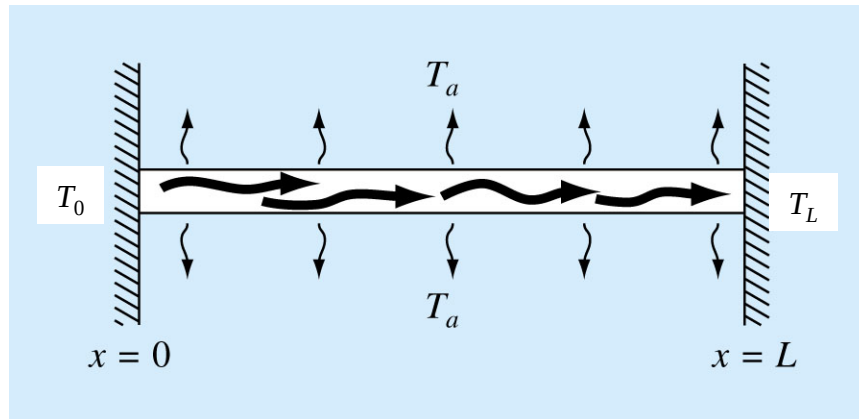
Nonlinear two-point BVP examples

- Nonlinear heat transfer.
- Simply supported beam undergoing large deflections.

Non-Linear BVP – Heat Transfer

$$\frac{d^2 T}{dx^2} + \alpha(T_a - T)^4 = 0; \quad T(0) = T_0, \quad T(L) = T_L$$

$$L = 10, \quad \alpha = 5 \times 10^{-8}, \quad T_a = 20, \quad T_0 = 40, \quad T_L = 200$$



Non-Linear BVP – Heat Transfer

- For matlab function `bvp4c` and shooting method transform using ($y_2 = dT/dx$, $y_1 = T$):

$$\frac{dy_1}{dx} = y_2$$

$$\frac{dy_2}{dx} = \alpha(y_1 - T_a)^4$$

$$y_1(0) = T_0, \quad y_1(L) = T_L$$

FDA with Newton's Method for Heat Example

- Non-linear Heat ODE

$$\frac{d^2T}{dx^2} + \alpha(T_{air} - T)^4 = 0; \quad T(0) = T_0, \quad T(L) = T_L$$

- FDA yields

$$E_i(\mathbf{T}) = T_{i-1} + \left(-2T_i - \alpha\Delta x^2 (T_i - T_a)^4 \right) + T_{i+1} = 0, \quad i = 1, \dots, n$$

Newton's Method For Heat Example

- Compute Jacobian matrix J is given by

$$J(\mathbf{T}^{(k)}) = \left(\frac{\partial E(\mathbf{T}^{(k)})}{\partial \mathbf{T}} \right)$$

- In each step k solve for residual

$$J(\mathbf{T}^{(k)}) \Delta \mathbf{T}^{(k)} = -E(\mathbf{T}^{(k)})$$

- Update new value of vector $\mathbf{T}$

$$\mathbf{T}^{(k+1)} = \mathbf{T}^{(k)} + \Delta \mathbf{T}^{(k)}$$

Jacobian Matrix for Heat Example

- The Jacobian matrix elements are given by

$$J_{ij} = \frac{\partial E_i}{\partial T_j}$$

$$E_i(\mathbf{T}) = T_{i-1} + \left(-2T_i - \alpha \Delta x^2 (T_i - T_a)^4 \right) + T_{i+1}$$

- The i^{th} row of the $n \times n$ Jacobian matrix J is given by (note that k is the known previous iterate)

$$J_{i,i-1} = \frac{\partial E_i}{\partial T_{i-1}} = 1$$

$$J_{i,i} = \frac{\partial E_i}{\partial T_i} = -2 - 4\alpha \Delta x^2 (T_i^{(k)} - T_a)^3$$

$$J_{i,i+1} = \frac{\partial E_i}{\partial T_{i+1}} = 1$$

RHS Vector for Heat Example

- Elements of the RHS vector are given by

$$b_i = -E(T_i^{(k)})$$
$$= -T_{i-1}^{(k)} + 2T_i^{(k)} + \alpha\Delta x^2 \left(T_i^{(k)} - T_a\right)^4 - T_{i+1}^{(k)}, \quad i = 2, \dots, n-1$$

- By substituting the known boundary values $T(0) = T_0$ and $T(L) = T_L$, rows of the rhs vector at $i = 1$ and $i = n$ can be modified as follows:

$$b_1 = -T_0 + 2T_1^{(k)} + \alpha\Delta x^2 \left(T_1^{(k)} - T_a\right)^4 - T_2^{(k)}$$

$$b_n = -T_{n-1} + 2T_n^{(k)} + \alpha\Delta x^2 \left(T_n^{(k)} - T_a\right)^4 - T_L$$

Newton's method for Heat Example (contd).

- Iterate until $\Delta \mathbf{T}^{(k)}$ becomes small enough or prescribed maximum number of iterations is reached ($k=0$ to maxiter).

$$\Delta \mathbf{T}^{(k)} = \mathbf{J}^{-1} \mathbf{b}$$

- Update new value of vector $\mathbf{T}^{(k+1)}$

$$\mathbf{T}^{(k+1)} = \mathbf{T}^{(k)} + \Delta \mathbf{T}^{(k)}$$

Non-Linear BVP – Beam

- The ODE is given by:

$$EI \frac{d^2 y}{dx^2} = -\frac{q}{2} \left(1 + \left(\frac{dy}{dx} \right)^2 \right)^{3/2} (x^2 - Lx); \quad y(0) = 0, \quad y(L) = 0$$

- For matlab function `bvp4c` and shooting method transform using ($y_2 = dy/dx$, $y_1 = y$):

$$\frac{dy_1}{dx} = y_2$$

$$\frac{dy_2}{dx} = -\frac{q}{2EI} \left(1 + y_2^2 \right)^{3/2} (x^2 - Lx)$$

$$y_1(0) = 0, \quad y_1(L) = 0$$

MATLAB implementation for nonlinear BVPs

- Nonlinear heat example is in:
 - [bvp_nonlinear_heat_example.m](#)
- Beam example is not provided here as it is part of assignment 4.

Differential Algebraic Equation (DAE) Systems

- Combination of ODEs and algebraic systems
 - E.g., Kinetic and Equilibrium equations in a reactor system
- Use MATLAB solvers `ode15s` or `ode15i` or `ode23t`
 - Inputs are slightly different for each solver

DAE System

- General form

$$\frac{d\mathbf{y}}{dx} = \mathbf{f}(x, \mathbf{y})$$

$$0 = \mathbf{g}(x, \mathbf{y})$$

- If the number of dependent variables is n (i.e., size of $\mathbf{y}$) then the number of ODEs+AEs is typically n
- Differential index – number of differentiations needed to transform the DAE system into an ODE system

DAE example – Robertson Problem

- DAE system

$$\frac{dy_1}{dt} = -0.04y_1 + 10^4 y_2 y_3$$

$$\frac{dy_2}{dt} = 0.04y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2$$

$$0 = y_1 + y_2 + y_3 - 1$$

- Initial conditions

$$y_1 = 1, \quad y_2 = 0, \quad y_3 = 0 \quad \text{at } t = 0$$

DAE solution in MATLAB

- ode15s is the preferred solver
- you can also use ode15i or ode23s but these require several intermediate steps (see examples in MATLAB help)
- For ode15s, you will need to set mass matrix and other options using odeset function
- If DAE index is greater than 1 then more elaborate steps need to be followed (see documentation for more details)
 - reduce index to 0 or 1 using the symbolic toolbox and use ode15s, ode15i, or ode23s